Digital Health Record Management System (DHRMS)

LAB 3: UML Class Diagram Design — Entity Modeling, Attributes, Methods & Relationships

Course: Software Engineering (SWE) Academic Session 2026–2027	**Faculty Guide:** Mr. Navdesh Singh, Dept. of CSE	**Source Baseline:** DHRMS SRS v2.0 (ISO/IEC/IEEE 29148)

1. Laboratory Objectives and Structural System Context

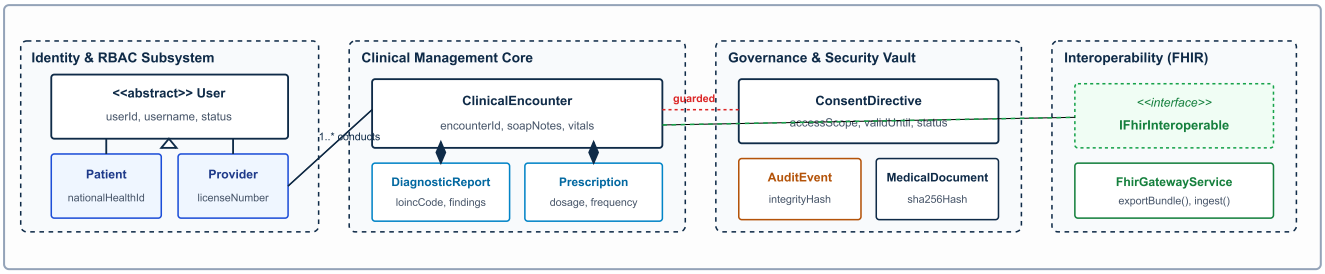
LAB DELIVERABLE 3

The objective of this laboratory is to systematically derive the structural Object-Oriented design for DHRMS from the functional requirements (REQ-AUTH, REQ-REC, REQ-ENC, REQ-DIAG, REQ-TREAT, REQ-CONSENT, REQ-DOC, REQ-AUDIT, REQ-INT) and use cases UC-01 through UC-10. This specification establishes domain entities, field encapsulations, visibility rules, typing contracts, operations, and inter-class structural coupling.

> **Architectural Invariant & Separation of Concerns:** As mandated by SRS v2.0 Business Rules BR-01 through BR-05, all clinical entities are encapsulated behind server-side authorization and active dynamic consent. Administrative roles manage system identity without direct access to clinical attributes, while the audit subsystem operates via immutable, append-only hashing.

2. UML Class Architecture & Package Interaction Overview

MODEL OVERVIEW



UML Modeling Conventions:

- **Visibility:** + Public, - Private, # Protected.
- **Generalization:** Solid line with hollow triangle (`User` inheritance).
- **Composition:** Solid line with filled diamond (Encounter owns Items).

Subsystem Coupling:

- **Realization:** Dashed line with hollow triangle to `IFhirInteroperable`.
- **Dynamic Interception:** Encounters check `ConsentDirective` via Strategy pattern prior to clinical data materialization.

3. Class Specifications: Identity, Role-Based Access & Actor Hierarchy

CLASSES 1 – 6

The table below outlines attributes, data types, visibility, method signatures, and role-based encapsulation for the primary actor and authentication hierarchy in compliance with **REQ-AUTH-001–008** and **REQ-RBAC-001–007**.

Class & Stereotype	Attributes (Visibility & Types)	Methods & Operations (Signature & Return)
User `<<abstract>>`	- userId: UUID - username: String - email: String - passwordHash: String - mfaSecret: String - status: AccountStatus	+ authenticate(password: String): Boolean + verifyMFA(totpCode: String): Boolean + resetPassword(newHash: String): Void + updateAccountStatus(status: AccountStatus): Void
Role `<<entity>>`	- roleId: Integer - roleName: RoleType - permissionMask: Long - description: String	+ hasPermission(requiredBit: Long): Boolean + assignPermission(permissionBit: Long): Void + revokePermission(permissionBit: Long): Void
Patient `extends User`	- patientId: UUID - nationalHealthId: String - fullName: String - dob: LocalDate - bloodType: BloodGroup - emergencyContact: String	+ viewHealthProfile(): PatientRecordDTO + grantConsent(pId: UUID, scope: Scope, exp: Date): ConsentDirective + revokeConsent(consentId: UUID): Boolean + viewAccessAuditLog(): List<AuditEventDTO>
HealthcareProvider `extends User`	- providerId: UUID - licenseNumber: String - specialization: String - department: String - hospitalAffiliation: String	+ searchPatient(nhid: String): PatientSummaryDTO + recordEncounter(pId: UUID, enc: EncounterDTO): UUID + createPrescription(encId: UUID, rx: RxDTO): UUID + orderDiagnosticReport(encId: UUID, loinc: String): UUID
SystemAdministrator `extends User`	- adminId: UUID - adminLevel: Integer - assignedZone: String	+ createAccount(userDTO: UserDTO): UUID + lockUserAccount(userId: UUID, reason: String): Void + configureSystemTelemetry(cfg: ConfigDTO): Void
ComplianceAuditor `extends User`	- auditorId: UUID - jurisdiction: String - certificationNumber: String	+ queryAuditTrail(filter: AuditFilterDTO): List<AuditEvent> + exportRegulatoryReport(period: DateRange): FileStream + verifyTamperResistance(): AuditVerificationResult

4. Class Specifications: Clinical Encounter Core Entities

CLASSES 7 – 8

Class & Stereotype	Attributes (Visibility & Types)	Methods & Operations (Signature & Return)
ClinicalEncounter `<<entity, aggregate root>>`	- encounterId: UUID - patientId: UUID - providerId: UUID - encounterDate: LocalDateTime - encounterType: EncounterType - chiefComplaint: String - soapNotes: String - vitals: VitalSigns	+ recordVitals(vitals: VitalSigns): Void + appendSoapNotes(notes: String): Void + addDiagnosis(icd10Code: String): Void + closeEncounter(): Void + generateClinicalSummary(): EncounterSummaryDTO + toFhirResource(): FhirEncounter
VitalSigns `<<value object>>`	- systolicBP: Integer - diastolicBP: Integer - heartRate: Integer - respiratoryRate: Integer - bodyTemperature: Double - oxygenSaturation: Double	+ isHypertensive(): Boolean + isFebrile(): Boolean + formattedBloodPressure(): String + validateMetricRanges(): Boolean

5. Class Specifications: Clinical Artifacts, Security & Interoperability

CLASSES 9 – 13

Class & Stereotype	Attributes (Visibility & Types)	Methods & Operations (Signature & Return)
DiagnosticReport `<<entity>>`	- reportId: UUID - encounterId: UUID - patientId: UUID - testCodeLoinc: String - findingsText: String - resultValue: Double - unit: String - labDate: LocalDateTime	+ evaluateReferenceRange(): ObservationStatus + attachFileMetadata(docId: UUID): Void + toFhirObservation(): FhirDiagnosticReport
Prescription `<<entity>>`	- prescriptionId: UUID - encounterId: UUID - patientId: UUID - medicationName: String - dosage: String - frequency: FrequencyType - durationDays: Integer - status: PrescriptionStatus	+ issuePrescription(): Void + markAsDispensed(pharmacyId: UUID): Void + cancelPrescription(reason: String): Void + isExpired(): Boolean
ConsentDirective `<<security entity>>`	- consentId: UUID - patientId: UUID - providerId: UUID - accessScope: ConsentScope - validFrom: LocalDateTime - validUntil: LocalDateTime - status: ConsentStatus	+ isActive(): Boolean + satisfiesScope(requestedScope: ConsentScope): Boolean + modifyScope(newScope: ConsentScope): Void + revoke(): Void
MedicalDocument `<<entity>>`	- documentId: UUID - patientId: UUID - uploaderId: UUID - fileName: String - storagePathS3: String - mimeType: MimeType - fileSize: Long - sha256Hash: String	+ verifyIntegrity(computedHash: String): Boolean + generatePresignedUrl(expiryMinutes: Int): String + validateMimeHeader(): Boolean
AuditEvent `<<immutable audit>>`	- auditId: Long - actorUserId: UUID - actionType: ActionType - resourceUri: String - clientIp: String - timestamp: LocalDateTime - outcome: OperationOutcome - integrityHash: String	+ computeHash(prevHash: String): String + verifyTamper(): Boolean + exportFormattedJson(): String

6. Class Relationship, Multiplicity & Dependency Specification

UML ASSOCIATION MATRIX

Source Class	Target Class	Relationship Type	Multiplicity	Business Rule / Architectural Invariant
User	**Role**	Directed Association	* to 1..*	Users must have at least one valid system role (REQ-RBAC-001).
Patient	**User**	Generalization (Inheritance)	1 to 1	Patient inherits credentials, MFA, and profile attributes.
HealthcareProvider	**User**	Generalization (Inheritance)	1 to 1	Provider inherits system authentication and identity properties.
Patient	**ConsentDirective**	Aggregation	1 to 0..*	Patient owns and manages dynamic provider consent directives.
HealthcareProvider	**ClinicalEncounter**	Association	1 to 0..*	Provider conducts consultations under verified consent (BR-02).
ClinicalEncounter	**DiagnosticReport**	Composition	1 to 0..*	Reports belong strictly to the clinical encounter context.
ClinicalEncounter	**Prescription**	Composition	1 to 0..*	Prescriptions are generated directly during an encounter session.

7. Object-Oriented Design Patterns Applied

ARCHITECTURE PATTERNS

To ensure high cohesion, low coupling, and strict compliance with healthcare regulatory benchmarks (HIPAA, OWASP ASVS Level 2), the DHRMS UML model incorporates standard GoF patterns:

Design Pattern	Classes Involved	Architectural Purpose & Impact
Strategy Pattern	`ConsentDirective`, `AccessPolicyEvaluator`	Enables dynamic runtime evaluation of granular patient consent scopes (e.g., ALL vs. DIAGNOSTIC_ONLY) before querying clinical data.
Factory Pattern	`FhirResourceFactory`, `IFhirInteroperable`	Encapsulates translation of internal clinical models into standardized HL7 FHIR R5 JSON structures (Patient, Encounter, Observation).
Observer Pattern	`AuditEventDispatcher`, `AuditEvent`	Asynchronously records security and PHI access events into append-only tables without blocking critical clinical response threads.
Repository Pattern	`PatientRepository`, `EncounterRepository`	Decouples business domain entities from underlying PostgreSQL 16 persistence schemas and S3 encrypted object storage handlers.

8. Functional Requirements & Use Case Traceability Matrix

TRACEABILITY

Use Case ID	Primary Domain Classes	SRS Requirements Covered	Realization Method
UC-01	`User`, `Role`, `IAuthService`	REQ-AUTH-001–008, REQ-RBAC-001	`authenticate()`, `verifyMFA()`
UC-02	`Patient`, `ClinicalEncounter`	REQ-REC-001–008, REQ-RBAC-003	`viewHealthProfile()`, `satisfiesScope()`
UC-03	`Patient`, `ConsentDirective`	REQ-CONSENT-001–008	`grantConsent()`, `revoke()`
UC-04	`HealthcareProvider`, `Encounter`	REQ-ENC-001–005, REQ-RBAC-004	`recordEncounter()`, `closeEncounter()`
UC-05, UC-06	`DiagnosticReport`, `Prescription`	REQ-DIAG-001–006, REQ-TREAT-001–005	`orderDiagnosticReport()`, `issuePrescription()`
UC-07, UC-09	`MedicalDocument`, `AuditEvent`	REQ-DOC-001–006, REQ-AUDIT-001–007	`verifyIntegrity()`, `computeHash()`

9. Academic Evaluation & Supervisory Sign-Off

EVALUATION

Faculty Guide Evaluation Remarks:

The UML Class Diagram specifications, attribute encapsulations, method typing, and multiplicity contracts conform to ISO/IEC/IEEE 29148 standards and the DHRMS Version 2.0 technical baseline. The design satisfies object-oriented decomposition principles and security isolation rules.

Status: APPROVED **Lab Grade:** _____ / 10

Faculty Supervisor:
Mr. Navdesh Singh
Dept. of Computer Science & Engg.

Faculty Signature & Date

Prepared by: Abhinav Singh (202401100200014), Abhinav Pathak (202401100200013), Abhishek Kumar Verma (202401100200015), Abhishek Yadav (202401100200016)